

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Северный (Арктический) федеральный университет имени М.В. Ломоносова»

Высшая школа информационных технологий и автоматизированных систем
(наименование высшей школы / филиала / института / колледжа)

ОТЧЕТ
о лабораторном практикуме

По дисциплине DevTestOps

На тему Тестирование ПО на python

Выполнил обучающийся:

Груздев Николай Александрович
(Ф.И.О.)

Направление подготовки / специальность:

09.03.01 Информатика и вычислительная
техника
(код и наименование)

Курс: 4

Группа: 151220

Руководитель: Тарасов А.П., старший
преподаватель кафедры информационных
систем и информационной безопасности САФУ
(Ф.И.О. руководителя, должность / уч. степень / звание)

Отметка о зачете

(отметка прописью)

(дата)

Руководитель

(подпись руководителя)

А.П. Тарасов
(инициалы, фамилия)

Архангельск 2025

ЛИСТ ДЛЯ ЗАМЕЧАНИЙ

ЦЕЛЬ РАБОТЫ

Получить практические навыки по тестированию приложения на python.

1 РЕШЕНИЕ ПОСТАВЛЕННОЙ ЗАДАЧИ

Установим библиотеки `pytest` и `pytest-cov` для организации тестирования модуля `data.py` и оценки покрытия кода (рис. 1-2).

```
[redos@redos Crystal code (test)]$ pip install pytest
Defaulting to user installation because normal site-packages is not writeable
Collecting pytest
  Downloading pytest-9.0.2-py3-none-any.whl.metadata (7.6 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Downloading iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: packaging>=22 in /usr/lib/python3.11/site-packages (from pytest) (25.0)
Collecting pluggy<2,>=1.5 (from pytest)
  Downloading pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest)
  Downloading pygments-2.19.2-py3-none-any.whl.metadata (2.5 kB)
Downloading pytest-9.0.2-py3-none-any.whl (374 kB)
Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Downloading iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Downloading pygments-2.19.2-py3-none-any.whl (1.2 MB)
```

Рисунок 1 – Установка `pytest`

```
[redos@redos Crystal code (test)]$ pip install pytest-cov
Defaulting to user installation because normal site-packages is not writeable
Collecting pytest-cov
  Downloading pytest_cov-7.0.0-py3-none-any.whl.metadata (31 kB)
Collecting coverage>=7.10.6 (from coverage[toml]>=7.10.6->pytest-cov)
  Downloading coverage-7.13.0-cp311-cp311-manylinux1_x86_64.manylinux2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (8.5 kB)
Requirement already satisfied: pluggy>=1.2 in /home/redos/.local/lib/python3.11/site-packages (from pytest-cov) (1.6.0)
Requirement already satisfied: pytest>=7 in /home/redos/.local/lib/python3.11/site-packages (from pytest-cov) (9.0.2)
Requirement already satisfied: iniconfig>=1.0.1 in /home/redos/.local/lib/python3.11/site-packages (from pytest>=7->pytest-cov) (2.3.0)
Requirement already satisfied: packaging>=22 in /usr/lib/python3.11/site-packages (from pytest-cov) (25.0)
```

Рисунок 2 – Установка `pytest-cov`

Создадим файл `test_data.py`, содержащий тесты для ключевых классов модуля `data.py`. Тесты охватывают не все классы, так как некоторые из них (например, `ListOfExaminations`) тесно связаны с другими (например, `Examination`), а в `ExaminationTemplates` недостаточно логики для проверки. Тесты выполняют методы классов и проверяют результаты с помощью запросов к базе данных (листинг 1).

Листинг 1 – `test_data.py`

```
from data import People, ListOfExaminations, Settings, Database,
Templates, ExaminationTemplates, Person, Examination
import os.path, datetime
def test_Database_class():
    DB = Database()
    DB.do_backup_of_database()
    now = datetime.datetime.now()
```

```

    assert
    os.path.isfile(f"data/data_{now.strftime('%y%m%d_%H%M')}.db") == True
    os.remove("data/data.db")
    DB = Database()
    assert DB.execute("SELECT On_top FROM Settings")
def test_Person_class():
    DB = Database()
    person = Person('Ivan Sergeevich Ozhigin', '12.12.2000', db=DB)
    assert person.full_name == 'Ivan Sergeevich Ozhigin' and
person.birthdate == '12.12.2000'
    person.add_to_database()
    db_full_name = DB.execute(f"SELECT Full_name FROM People WHERE Id
= {person.id}").fetchone()[0]
    db_birthdate = DB.execute(f"SELECT Birthdate FROM People WHERE Id
= {person.id}").fetchone()[0]
    assert person.full_name == db_full_name and person.birthdate ==
db_birthdate
    person = Person('Ivan Sergeevich Ozhigin', '15.07.2001', person.id
,DB)
    person.update_data_in_database()
    db_birthdate = DB.execute(f"SELECT Birthdate FROM People WHERE Id
= {person.id}").fetchone()[0]
    assert person.birthdate == db_birthdate
    person.delete_from_database()
    assert DB.execute(f"SELECT EXISTS(SELECT Id FROM People WHERE Id =
{person.id})").fetchone()[0] == False
def test_People_class():
    DB = Database()
    people = People(DB)
    new_p = people.add_person('COOLMAN254', '05.08.2007')
    db_full_name = DB.execute(f"SELECT Full_name FROM People WHERE Id
= {new_p.id}").fetchone()[0]
    db_birthdate = DB.execute(f"SELECT Birthdate FROM People WHERE Id
= {new_p.id}").fetchone()[0]
    assert new_p.full_name == 'COOLMAN254' and new_p.birthdate ==
'05.08.2007'
    assert new_p.full_name == db_full_name and new_p.birthdate ==
db_birthdate
    found = people.search(id=new_p.id)
    assert found.full_name == 'COOLMAN254'
    found = people.search(name='COOLMAN254')
    assert found[0].full_name == 'COOLMAN254'
    found = people.search(id=new_p.id, name='COOLMAN254')
    assert found == None
    found = people.search()
    assert found == None
    people.update_person_data(new_p.id, 'COOLMAN2546', '05.10.2007')
    db_full_name = DB.execute(f"SELECT Full_name FROM People WHERE Id
= {new_p.id}").fetchone()[0]
    db_birthdate = DB.execute(f"SELECT Birthdate FROM People WHERE Id
= {new_p.id}").fetchone()[0]
    assert db_full_name == 'COOLMAN2546' and db_birthdate ==
'05.10.2007'
    p_list = people.get_all()
    for p in p_list:
        if p.id == new_p.id:

```

```

        assert p.full_name == 'COOLMAN2546' and p.birthdate ==
'05.10.2007'
        break
    people.delete_person(new_p.id)
    assert DB.execute(f"SELECT EXISTS(SELECT Id FROM People WHERE Id =
{new_p.id})").fetchone()[0] == False
def test_Examination_class():
    DB = Database()
    people = People(DB)
    new_p = people.add_person('COOLMAN254', '05.08.2007')
    le = ListOfExaminations(DB)
    le.add_examination(person_id=new_p.id,
                        eyesight_without_od=1.0,
                        eyesight_without_os=1.0,
                        eyesight_with_od=1.0,
                        eyesight_with_od_sph=0.0,
                        eyesight_with_od_cyl=0.0,
                        eyesight_with_od_ax=0,
                        eyesight_with_os=1.0,
                        eyesight_with_os_sph=0.0,
                        eyesight_with_os_cyl=0.0,
                        eyesight_with_os_ax=0,
                        schiascopy_od='Em',
                        schiascopy_os='Em',
                        glasses_od_sph=0.0,
                        glasses_od_cyl=0.0,
                        glasses_od_ax=0,
                        glasses_os_sph=0.0,
                        glasses_os_cyl=0.0,
                        glasses_os_ax=0,
                        glasses_dpp=0,
                        diagnosis_subscription='',
                        visit_date='23.03.2025 13:09',
                        complaints='',
                        disease_anamnesis='',
                        life_anamnesis='',
                        eyesight_type='',
                        relative_accommodation_reserve=-1.0,
                        schober_test='',
                        pupils='',
                        od_eye_position='ортофория',
                        od_oi='спокойны',
                        od_eyelid='без изменений',
                        od_lacrimonal_organs='без изменений',
                        od_conjunctiva='бледная, розовая',
                        od_discharge='отсутствует',
                        od_iris='структура сохранена, рисунок четкий',
                        od_anterior_chamber='глубина обычная',
                        od_refractive_medium='прозрачные',
                        od_optic_disk='бледно-розовый, границы
четкие',
                        od_vessels='норма',
                        od_macular_reflex='сохранен',
                        od_visible_periphery='без особенностей',
                        od_diagnosis='Здоров',
                        od_icd_code='u13.1',
                        os_eye_position='ортофория',

```

```

        os_oi='спокойны',
        os_eyelid='без изменений',
        os_lacrimonal_organs='без изменений',
        os_conjunctiva='бледная, розовая',
        os_discharge='отсутствует',
        os_iris='структура сохранена, рисунок четкий',
        os_anterior_chamber='глубина обычная',
        os_refractive_medium='прозрачные',
        os_optic_disk='бледно-розовый, границы
четкие',

        os_vessels='норма',
        os_macular_reflex='сохранен',
        os_visible_periphery='без особенностей',
        os_diagnosis='Здоров',
        os_icd_code='u13.2',
        recommendations='гигиена зрения',
        direction_to_aokb=0,
        reappointment=0,
        reappointment_time='',
    )

    e = le.get_examinations_by_person_id(new_p.id)
    assert e[0].visit_date == '23.03.2025 13:09'
    assert e[0].os_iris == 'структура сохранена, рисунок четкий'
    e = le.get_examination_by_id(e[0].id)
    assert e.os_iris == 'структура сохранена, рисунок четкий'
    e =
le.get_examination_by_person_id_and_examination_datetime(e.person_id,
e.visit_date)
    assert e.os_icd_code == 'u13.2'
    le.update_examination(e.id, person_id=new_p.id,
        eyesight_without_od=1.0,
        eyesight_without_os=1.0,
        eyesight_with_od=1.0,
        eyesight_with_od_sph=0.0,
        eyesight_with_od_cyl=0.0,
        eyesight_with_od_ax=0,
        eyesight_with_os=1.0,
        eyesight_with_os_sph=0.0,
        eyesight_with_os_cyl=0.0,
        eyesight_with_os_ax=0,
        schiascopy_od='Em',
        schiascopy_os='Em',
        glasses_od_sph=0.0,
        glasses_od_cyl=0.0,
        glasses_od_ax=0,
        glasses_os_sph=10.0,
        glasses_os_cyl=0.0,
        glasses_os_ax=0,
        glasses_dpp=0,
        diagnosis_subscription='',
        complaints='')
    assert e.glasses_os_sph == 10.0
    ids = le.get_people_ids()
    assert (new_p.id in ids) == True
    le.delete_examination(e.id, people)
    ids = le.get_people_ids()
    assert (new_p.id in ids) == False

```

```
def test_Settings_class():
    DB = Database()
    s = Settings(DB)
    s.update_use_password(False)
    s = Settings(DB)
    assert s.use_password == False
```

Запустим тесты командой `pytest --cov` в директории проекта. В ходе выполнения обнаружена ошибка: в методе `add_person` класса `People` вместо возврата объекта `new_person` код пытается вернуть несуществующую переменную `new_persona` (рис. 3).

```
test_data.py:38:
-----
self = <data.People object at 0x7f2fced44f90>, full_name = 'COOLMAN254'
birthdate = '05.08.2007'

    def add_person(self, full_name: str, birthdate: str) -> Person:
        new_person = Person(full_name, birthdate, id=None, db=self.__db)
        self.__people[new_person.id] = new_person
        self.__people[new_person.id].add_to_database()
>       return new_persona
           ^^^^^^^^^^^^^
E       NameError: name 'new_persona' is not defined

data.py:1103: NameError
----- Captured stdout call -----
Set only one arg
Set one of the arg
===== short test summary info =====
FAILED test_data.py::test_People_class - NameError: name 'new_persona' is not defined
===== 1 failed, 1 passed in 0.44s =====
```

Рисунок 3 – Обнаруженная ошибка в процессе тестирования

Исправим опечатку в файле `data.py`, заменив `new_persona` на `new_person`. Повторно запустим тесты. Теперь все проверки проходят успешно, а отчёт о покрытии кода показывает 75% для модуля `data.py` (рис. 4). Тестирование программного обеспечения завершено.

```
[redos@redos Crystal code (test)]$ pytest --cov
===== test session starts =====
platform linux -- Python 3.11.14, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/redos/Crystal code (test)
plugins: cov-7.0.0
collected 5 items

test_data.py ..... [100%]

===== tests coverage =====
coverage: platform linux, python 3.11.14-final-0
Name      Stmts  Miss  Cover
-----
data.py    727    185    75%
test_data.py  77      0   100%
TOTAL      804    185    77%
===== 5 passed in 1.03s =====
```

Рисунок 4 – Отчёт об успешном выполнении тестов и покрытии кода

2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. `pytest` в сравнении с `unittest` отличается более лаконичным синтаксисом (достаточно простых функций и `assert`), высокой гибкостью и богатой экосистемой плагинов. В отличие от `unittest`, который является встроенным модулем, требует наследования от `TestCase` и использования методов вида `assertEqual()`, `pytest` автоматически находит тесты, поддерживает параметризацию и мощные фикстуры, что ускоряет разработку и делает код тестов проще для чтения;

2. Покрытие кода — это метрика, показывающая процент строк исходного кода, который был выполнен в процессе тестирования. Она помогает оценить, насколько полно тесты проверяют функционал программы;

3. При запуске `pytest` не требуется явно указывать файлы с тестами. Фреймворк автоматически обнаруживает и выполняет все файлы, имена которых соответствуют шаблонам `test_*.py` или `*_test.py` в текущей директории и её подкаталогах.

ВЫВОД

В ходе работы были приобретены практические навыки написания и выполнения автоматических тестов для Python-приложения с использованием фреймворка `pytest` и оценки покрытия кода.